

Replicated Concurrency Control & Recovery Project — Design Document

Team: Tejas Choudhary (tkc8441) and Vincent Xu (yx2021)

Problem Statement

The goal of this project is to simulate a distributed database system that guarantees Serializable Snapshot Isolation (SSI) while supporting data replication and failure recovery. The system consists of 10 sites and 20 data variables. Odd-indexed variables are stored at a single site, while even-indexed variables are replicated across all sites.

The core challenge is to implement the Available Copies algorithm in conjunction with SSI. The system must adhere to specific correctness rules:

- Readers must see a consistent snapshot of the database as of the transaction start time.
- If a site fails after a transaction writes to it but before the transaction commits, the transaction must abort.
- The system must detect and prevent anomalies allowed by standard Snapshot Isolation.
- Recovered sites must not serve stale data for replicated variables until they are brought up to date by a new commit.

Design Decisions

1. **Buffered Writes:** Instead of sending write operations directly to Data Managers as they occur, we buffer all writes in the Transaction Manager's local TxRecord. Actual data modification only happens during the commit phase (end(T)).

Justification: This simplifies the Data Manager implementation significantly. It prevents "dirty reads" by design and allows the Transaction Manager to perform global validation (checking for conflicts across all sites) before any persistent changes are made. If validation fails, we simply discard the buffer, requiring no complex rollback logic at the sites.

2. **Lazy Serialization Graph Cleanup:** Transactions are removed from the serialization graph only when no active transactions remain that could potentially depend on them.

Justification: This ensures that we never delete a node that might be part of a cycle for a currently active transaction thus guaranteeing correctness.

3. **Stale Variable Handling on Recovery:** When a site recovers, we mark all its replicated variables as "stale" in a specific staleVariables map within the Transaction Manager.

Justification: A recovering site may have missed writes while it was down. Serving a read from this site would violate consistency. We keep the variable stale until a new write commits to it on that site. This is a simpler alternative to a full catching up phase (copying data from other sites) and satisfies the requirement that reads must fail or block if no valid copy is available.

High Level Design

The system architecture follows a centralized control model to simplify the coordination of distributed sites. It consists of three main layers:

The Transaction Manager (Control Plane)

This is the central coordinator. It receives commands (parsed by the Driver), manages the lifecycle of transactions, routes operations to appropriate sites.. It maintains the global logical clock and the Serialization Graph for conflict detection.

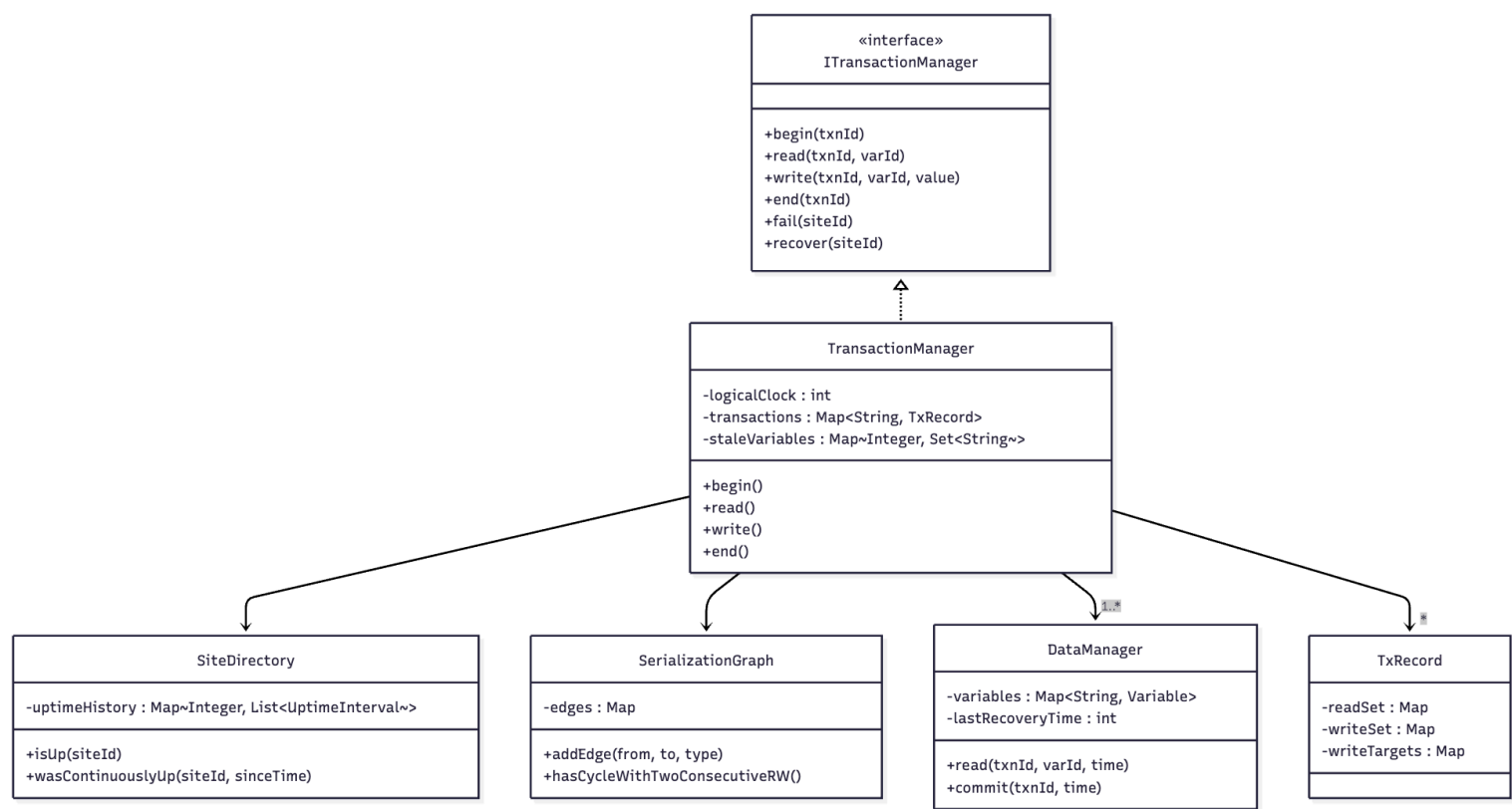
The Data Managers (Data Plane)

There are 10 independent Data Managers, one for each site. To support SSI, they implement Multiversion Concurrency Control (MVCC), storing chains of versioned values (value + commit timestamp) for each variable. They do not communicate with each other; all coordination is handled by the Transaction Manager.

The Site Directory

This is a specialized component within the Transaction Manager that acts as the source of truth for site availability. It tracks the historical uptime and downtime intervals of every site. This component is critical for the Available Copies algorithm, specifically for validating the rule: *a transaction may only commit if the sites it wrote to have been continuously up since the write occurred.*

High level class diagram



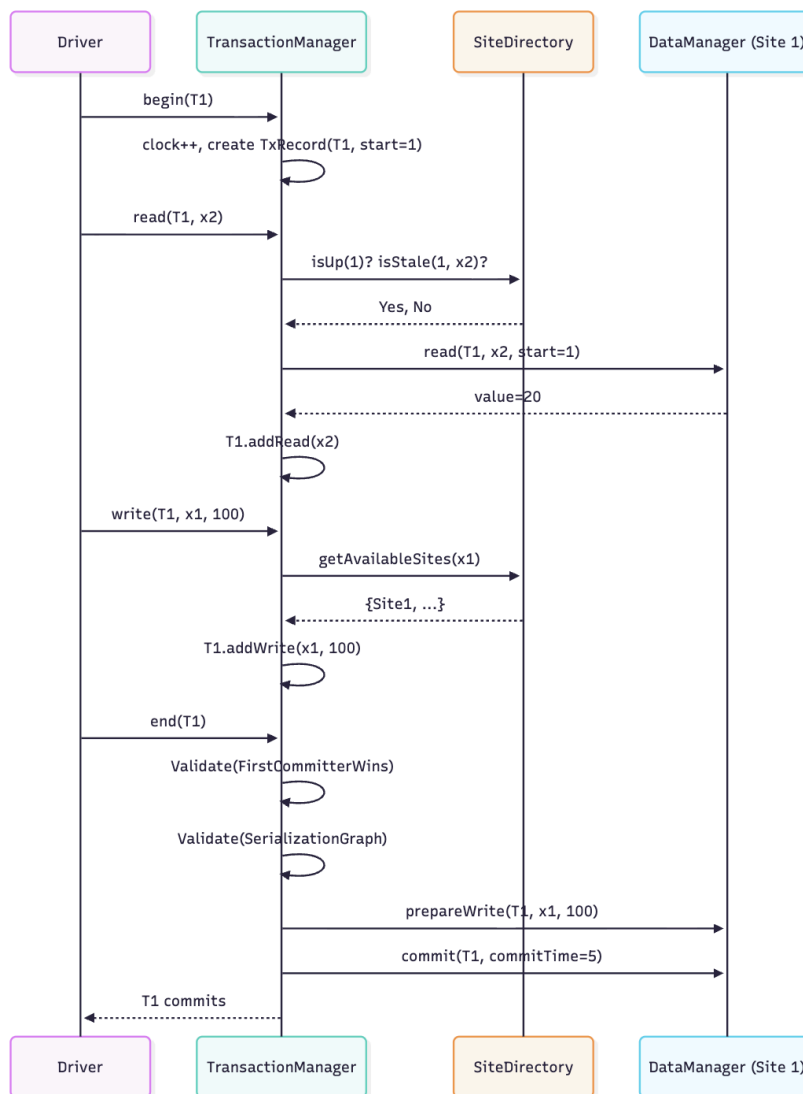
Low Level Design

Transaction Manager

The TransactionManager class implements the core logic.

- `begin(txnId)`: Increments the logical clock and creates a new `TxRecord`.
- `read(txnId, varId)`: Consults the `SiteDirectory` to find a valid site. A site is valid if it is UP and, for replicated variables, if it does not contain stale data and has been continuously UP since the transaction start.
- `write(txnId, varId, value)`: Buffers the write intent. Crucially, it captures the set of currently UP sites at the moment of the write. This captured set is used later during validation.
- `end(txnId)`: This is the most complex operation. It performs three validation steps:
 - Available Copies Validation: Verifies that sites targeted by writes did not fail.
 - First-Committer-Wins: Checks if any concurrent committed transaction wrote to the same variables.
 - Serialization Graph Validation: Checks for cycles containing two consecutive RW edges using DFS. If all pass, it flushes writes to the Data Managers using a two-phase prepare / commit sequence.

Sequence Diagram - Successful Transaction



Data Manager

The DataManager class manages Variable objects.

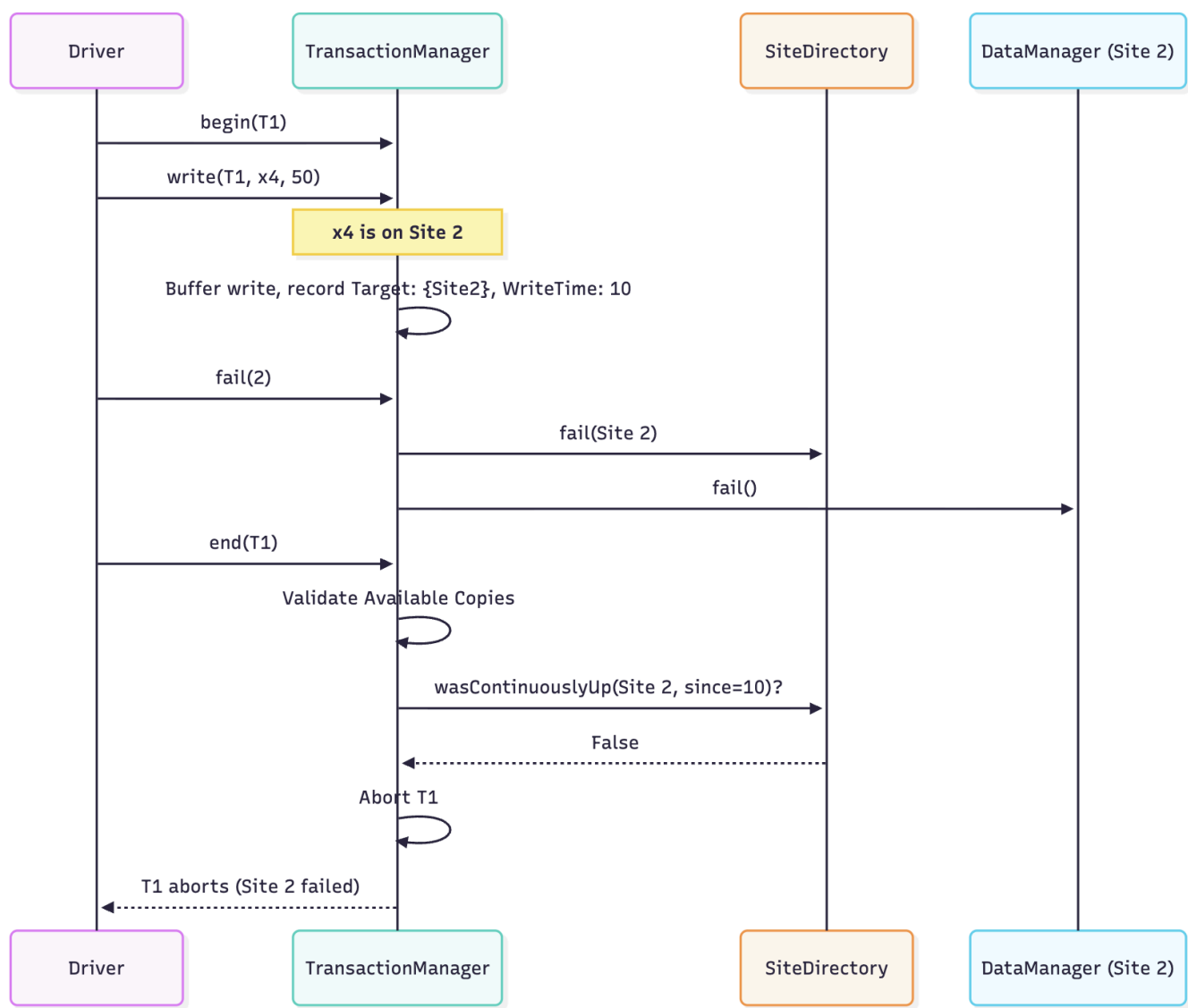
- read(varId, timestamp): Returns the latest version of a variable committed before or at the given timestamp.
- prepareWrite / commit: Allows the TM to persist new versions.
- fail / recover: Simulates failure by clearing volatile state (though in our simulation, committed data persists).

Serialization Graph

The SerializationGraph class maintains an adjacency list of dependencies.

- **Edge Types:** We track WW (Write-Write), WR (Write-Read), and RW (Read-Write/Anti-dependency) edges.
- **Cycle Detection:** A custom DFS algorithm searches specifically for cycles that contain at least two consecutive RW edges, which is the specific anomaly signature for non-serializable executions under Snapshot Isolation.

Sequence Diagram - Failure Handling



Code Structure

The project is organized into the com.ads package:

- com.ads.interfaces: Contains ITransactionManager and IDataManager to decouple implementation from the driver.
- TransactionManager.java: The core orchestration logic.
- DataManager.java: Site-specific storage logic.
- SiteDirectory.java: Logic for tracking site uptime history.
- SerializationGraph.java: Graph algorithms for SSI validation.
- TxRecord.java: A POJO holding the state of a single transaction (read set, write set, start time, status).
- Main.java: The entry point that parses standard input/files and drives the Transaction Manager.

Testing Strategy

Integration Testing: The Main class allows us to run full test scripts defined in the project description.